CSE 711 — Compiler

All Non-Numerical (Theory) Questions — Model Answers

Past papers 2016–2024 · University of Chittagong · Exam: Wed 24 Jun 2026

Scope. This sheet answers every *conceptual / written-answer* question (definitions, comparisons, explanations) found across the 8-year question map. It deliberately *excludes* the construction & computation problems (derivations, ambiguity demos, left-recursion elimination, left-factoring, FIRST/FOLLOW, LL(1)/LR table construction, parse traces, annotated parse trees, dependency graphs, DAGs, TAC/quad/triple generation, activation-record diagrams, CFG construction, instruction-cost calculation) — those need worked diagrams, not prose. Year tags **[20XX]** show where each concept was asked.

Contents

Lexical Analysis

Token, Pattern, Lexeme [2024 Q2c] [2022 A-2] [2021 Q3a] [2018 Q2a] [2017 Q1b] [2016 Q2a]

- **Token:** a pair ⟨ token-name, optional attribute value ⟩. The token-name is an abstract symbol (a terminal of the grammar) the parser works with, e.g. `id`, `num`, `if`, `relop`.
- **Pattern:** the rule — usually a regular expression — that describes the *set* of strings a token may stand for.
- **Lexeme:** the actual sequence of characters in the source program that matches a pattern and is recognised as one instance of a token.

Relationship: the scanner reads characters and forms a *lexeme*; the lexeme matches a *pattern*; the pattern defines a *token* that is passed to the parser.

Token	Pattern	Sample lexemes
`id`	letter followed by letters/digits	`count`, `x1`, `position`
`num`	digit$^+$ (with optional fraction/exp)	`27`, `3.14`
`relop`	`< | <= | = | <> | > | >=`	`<=`, `>`
`if`	the characters `i f`	`if`

Scanning vs. Parsing; Lexer/Parser Tools [2024 Q2d] [2023 A-4a]

	Scanning (Lexical Analysis)	Parsing (Syntax Analysis)
Job	groups characters into tokens; strips whitespace/comments	groups tokens into grammatical phrases; builds the parse tree
Governed by	regular expressions / DFA	context-free grammar / pushdown automaton
Power	regular languages	context-free languages
Output	stream of tokens	parse / syntax tree
Errors caught	illegal characters, malformed tokens	wrong token order, missing `)`, `;`

Tools. **LEX / Flex** generates a scanner from regular-expression rules; **YACC / Bison** generates a parser from a context-free grammar with semantic actions. They are used together: LEX feeds tokens to the YACC-generated parser.

Symbol Table — Role, Components, Hash Implementation [2023 A-1c]

Role. A data structure that records information about every identifier (name) in the program. It is built by the lexer/parser and consulted by *every* later phase for scope resolution, type checking, and address/code generation.

Components of an entry:

- name (the lexeme), token/kind (variable, function, type…)
- data type and storage class / scope
- size and memory offset / address
- line of declaration; for functions: parameter list & return type

Hash-table implementation (C). An array of buckets indexed by a hash of the name; collisions resolved by *chaining* (each bucket is a linked list of entries). `insert` and `lookup` are $O(1)$ on average.

Input Buffering [2017 Q1b]

To read source efficiently the scanner uses a **two-buffer scheme**: two halves of size N filled by one system read each. Two pointers — `lexemeBegin` (start of current lexeme) and `forward` (scans ahead). A **sentinel** (a special `eof` char) is placed at the end of each half so a *single* test on `forward` detects both "end of buffer" and "end of file," minimising comparisons per character.

Top-Down Parsing

Top-Down vs. Bottom-Up Parsing [2024 Q4a]

	Top-Down	Bottom-Up
Tree built	root → leaves	leaves → root
Derivation	*leftmost*	*rightmost*, in reverse
Technique	predictive / recursive-descent (LL)	shift-reduce (LR family)
Decision	expands a nonterminal using lookahead	reduces a *handle* on the stack
Left recursion	not allowed (infinite loop)	no problem
Grammar class	smaller (LL $\subset$ LR)	larger / more powerful

Recursive-Descent Parsing — How It Works [2024 Q4b] [2022 A-4] [2016 Q3a]

- One *procedure per nonterminal*. The procedure body mirrors the right-hand side: it *matches* terminals against the current input and *calls* the procedure of each nonterminal it meets.
- **General** recursive descent may need *backtracking* (try an alternative, undo input on failure).
- **Predictive** recursive descent uses one lookahead token to choose the alternative with *no* backtracking — it requires a grammar that is left-factored and free of left recursion.
- **Issue:** a left-recursive production $A \to A\alpha$ makes the procedure call itself forever — left recursion must be eliminated first.

A procedure typically uses helpers `match()/scan()` (consume expected token) and `error()` (report & recover).

Ambiguity [2024 Q3d] [2022 A-3]

A grammar is **ambiguous** if some string in its language has *more than one* parse tree (equivalently more than one leftmost or more than one rightmost derivation). It is a problem because the parser cannot decide which structure — hence which meaning — to assign. Classic example: $E \to E + E \mid E * E \mid (E) \mid$ `id` gives two trees for `id+id*id`. Fixes: rewrite with precedence/associativity levels, or impose a disambiguating rule.

Dangling-Else Problem [2023 A-4b]

The grammar

$$\texttt{stmt} \to \texttt{if}\ E\ \texttt{then stmt} \mid \texttt{if}\ E\ \texttt{then stmt else stmt} \mid \texttt{other}$$

is ambiguous: in `if` E_1 `then if` E_2 `then` S_1 `else` S_2 the `else` can bind to either `if`. **Rule used in practice:** match each `else` with the *nearest unmatched* `if`. The disambiguated grammar separates *matched* and *open* statements:

$$\begin{aligned} \texttt{stmt} &\to \texttt{matched} \mid \texttt{open} \\ \texttt{matched} &\to \texttt{if}\ E\ \texttt{then matched else matched} \mid \texttt{other} \\ \texttt{open} &\to \texttt{if}\ E\ \texttt{then stmt} \mid \texttt{if}\ E\ \texttt{then matched else open} \end{aligned}$$

which forces every `else` onto the closest preceding `then`, removing the ambiguity.

Error-Recovery Strategies [2023 A-3b] [2022 A-2] [2020 Q1c] [2018 Q1a]

- **Panic mode:** on an error, skip input tokens until a *synchronising* token (e.g. `;`, `)`, `end`) appears, then resume. Simple, never loops.
- **Phrase-level:** perform a local correction on the remaining input (replace/insert/delete a few symbols) to continue.
- **Error productions:** add grammar rules for common mistakes so the parser can recognise and diagnose them.
- **Global correction:** find the *minimum* number of changes to turn the input into a valid string — theoretically ideal but too costly in practice.

In predictive (LL) parsing: use FOLLOW(A) tokens to *synchronise* (pop A and resume) and FIRST(A) tokens to decide what to skip.

LL vs. LR Parser [2022 B-1a]

	LL	**LR**
Scan	Left-to-right	Left-to-right
Derivation traced	Leftmost	Rightmost (reverse)
Direction	top-down (predictive)	bottom-up (shift-reduce)
Left recursion	cannot handle	handles it
Power	weaker: LL(1) $\subset$ LR(1)	more grammars accepted
Table	predictive parsing table	ACTION + GOTO tables

Why Left Recursion Is Not a Problem for Bottom-Up Parsing [2022 A-3]

A shift-reduce parser builds the tree from the *leaves up*: it shifts terminals onto the stack and reduces a *handle* the moment one appears on top. For $A \to A\alpha$ it simply shifts the leading symbols and reduces left-to-right, so the recursion terminates naturally. A top-down parser, by contrast, would re-enter A before reading any input and loop forever. Hence left recursion is harmless (even idiomatic) for bottom-up parsers, but fatal for top-down ones.

Suitability for Top-Down / LL(1) [2023 A-2a]

A grammar is fit for predictive top-down (LL(1)) parsing iff for every nonterminal A with alternatives $A \to \alpha \mid \beta$:

1. it has *no left recursion* and is *left-factored*;
2. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \varnothing$; and
3. if $\beta \Rightarrow^* \varepsilon$ then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \varnothing$.

These conditions guarantee a unique production choice from one lookahead token.

Bottom-Up Parsing

LR(0) vs. SLR(1) vs. LR(1) vs. LALR(1) [2024 Q6a]

Parser	Characteristics
LR(0)	no lookahead; reduces in any state holding $A \to \alpha\bullet$; weakest, most conflicts.
SLR(1)	LR(0) items + reduce by $A \to \alpha$ only when next token $\in$ FOLLOW(A); small tables, fewer conflicts.
LR(1) (canonical)	items carry an explicit lookahead symbol; most powerful; largest number of states/tables.
LALR(1)	merges LR(1) states with the same core; same #states as SLR, almost as strong as LR(1); used by YACC. May add reduce–reduce conflicts but never shift–reduce.

Power: LR(0) $\subset$ SLR(1) $\subset$ LALR(1) $\subset$ LR(1).

Shift-Reduce Parsing Conflicts [2023 B-1a]

- **Shift-reduce conflict:** in some state the parser can neither decide to *shift* the next token nor to *reduce* a completed production (classic case: the dangling-else).
- **Reduce-reduce conflict:** two different productions are both candidates for reduction in the same state on the same lookahead.

Both arise from ambiguous or non-LR grammars; resolving them needs more lookahead, grammar rewriting, or a stated precedence rule.

Which Parser Class Can Parse a Grammar [2017 Q5a]

To classify a grammar, test it bottom of the hierarchy upward and report the smallest class that succeeds:

$$\text{LL}(1) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{LR}(1).$$

Check LL(1) by the disjoint-FIRST/FOLLOW conditions (needs no left recursion, left-factored). Check the LR family by building item sets and looking for shift-reduce / reduce-reduce conflicts. An ambiguous grammar belongs to *none* of these deterministic classes.

Syntax-Directed Translation

Synthesized vs. Inherited Attributes [2024 Q6c] [2023 B-1b] [2022 B-1b] [2021 Q5] [2020 Q3b] [2018 Q2c] [2016 Q2c]

	Synthesized	**Inherited**
Computed from	attributes of the node's *children* (and itself)	attributes of *parent* and/or *siblings*
Information flow	bottom-up	top-down / left-to-right
Defined at	$A \to \alpha$, value of A from RHS symbols	a symbol on the RHS, value from parent/left siblings
Typical use	value of an expression, code of a subtree	passing a declared type down to a list of ids

A *terminal* has only synthesized attributes (supplied by the lexer); the *start symbol* has no inherited attributes.

S-Attributed vs. L-Attributed Definitions [2024 Q6c] [2023 B-3a] [2018 Q2c] [2017 Q4c] [2016 Q2c]

- **S-attributed SDD:** uses *only synthesized* attributes. It can be evaluated bottom-up during an LR parse (attributes computed on reduction).
- **L-attributed SDD:** every inherited attribute of a RHS symbol X_j depends only on the inherited attributes of the parent and on attributes of symbols *to the left* of X_j (plus any synthesized attributes). It can be evaluated in a single left-to-right, depth-first pass — compatible with LL (predictive) parsing.

Relationship: every S-attributed definition is also L-attributed (no inherited attributes to violate the rule), so S-attributed $\subset$ L-attributed.

Intermediate Code

Quadruples vs. Triples vs. Indirect Triples vs. SSA [2020 Q7c]

Form	Description
Quadruple	$(op,\ arg_1,\ arg_2,\ result)$ — results held in *explicit temporary names*. Easy to move/reorder for optimization; uses most space.
Triple	$(op,\ arg_1,\ arg_2)$ — a result is referenced by the *position (index)* of its triple. Compact, but reordering is hard because positions are the references.
Indirect triple	a separate *list of pointers* to the triples. Optimization reorders only the pointer list; the triples themselves stay fixed — best of both.
SSA	Static Single Assignment: every variable is assigned *exactly once*; ϕ-functions are inserted at control-flow merge points. Greatly simplifies data-flow optimizations.

Runtime Environments

Activation Record & Runtime Stack [2018 Q7c] [2018 Q8b]

An **activation record** (stack frame) is the storage block for *one activation* of a procedure. Typical fields (top to bottom):

- temporaries and local data
- saved machine state (return address, saved registers)
- **access (static) link** — to the enclosing scope, for non-local names
- **control (dynamic) link** — to the caller's activation record
- actual parameters and the returned value

The **runtime stack** holds these records: one is *pushed* on each call and *popped* on return. Because every call gets a fresh record, the stack naturally supports *recursion*.

Static vs. Dynamic Storage & Garbage Collection [2023 B-4]

- **Static allocation:** storage sizes/addresses fixed at compile time (globals, static data). No recursion, no run-time sizing.
- **Stack (dynamic) allocation:** activation records pushed/popped in LIFO order; supports recursion and block scope.

- **Heap allocation:** for data whose lifetime is unrelated to the call that created it (objects, `malloc`/`new`); allocated/freed in any order.

Garbage collector: reclaims heap storage that is no longer reachable from the program, automatically. Common techniques: *reference counting*, *mark-and-sweep*, and *copying* collection.

Environment, State, and Static Binding [2017 Q1a]

- **Environment:** a mapping from a *name* to a *storage location* (its l-value).
- **State:** a mapping from a *storage location* to the *value* held there (its r-value).

An assignment changes the *state*; entering a new scope or declaring a name changes the *environment*. **Static (compile-time) binding** fixes an association — such as scope or type — when the program is compiled; *dynamic binding* fixes it at run time.

Code Optimisation

Basic Block & Flow Graph [2024 Q7c]

- **Basic block:** a maximal sequence of consecutive instructions with a *single entry* (the first instruction) and a *single exit* (the last) — control enters only at the top and leaves only at the bottom, no internal branching.
- **Leaders** (first instruction of each block): the first instruction of the program; any jump target; and any instruction immediately following a jump.
- **Flow graph (CFG):** a directed graph whose nodes are basic blocks and whose edges show possible flow of control between them.

Objectives of Optimization [2024 B-3b]

Transform the intermediate/target code so it *runs faster and/or uses less memory*, while:

- **preserving the program's meaning** (semantics-preserving — the output must be identical);
- being worthwhile *on average* (the speed-up justifies compile-time cost);
- not degrading the common case to help a rare one.

Copy Propagation [2023 B-2c]

After a copy statement $x = y$, replace later *uses* of x by y wherever x is not redefined in between. **Advantage:** it often makes the copy statement *dead* (so dead-code elimination can delete it) and exposes further optimizations such as constant folding and common-subexpression elimination.

Optimization Techniques (names & meaning) [2024 B-3b] [2016 Q8]

Technique	What it does
Common-subexpression elimination	reuse a value already computed instead of recomputing it.
Copy propagation	replace uses of x by y after $x = y$.
Constant folding / propagation	evaluate constant expressions at compile time; spread known constants.
Dead-code elimination	remove code whose result is never used.
Strength reduction	replace a costly op by a cheaper one (e.g. $x * 2 \rightarrow x+x$, multiply $\rightarrow$ add in loops).
Code motion (loop-invariant hoisting)	move computations that don't change inside a loop to before the loop.
Induction-variable elimination	remove redundant loop counters derived from one another.
Algebraic simplification	apply identities such as $x + 0 = x$, $x * 1 = x$.

These are applied *locally* (within a basic block, often via a DAG) and *globally* (across the flow graph, using data-flow analysis).

Code Generation

Stack Frames; Caller- vs. Callee-Saved Registers [2021 Q7c]

Stack frame. The per-call region on the runtime stack holding the return address, saved registers, local variables and spilled temporaries; addressed via a *frame pointer* (`rbp`) with the *stack pointer* (`rsp`) at the top.

Register class	Who preserves it
Caller-saved (volatile)	the *caller* must save them before a call if it needs the values afterward; the callee may clobber them freely. *(x86-64 SysV: `rax`, `rcx`, `rdx`, `rsi`, `rdi`, `r8`–`r11`.)*
Callee-saved (non-volatile)	the *callee* must save and restore them, so the caller may assume they survive the call. *(`rbx`, `rbp`, `r12`–`r15`.)*

End of theory model answers — CSE 711 Compiler (2016–2024). Good luck.